My-Finance-Tracker

Implemented Features

Generated July 20, 2026

AI-Powered Personal Finance, Investment & Stock Analytics Platform

Stack: FastAPI + SQLAlchemy + SQLite (backend) | React + TypeScript + Vite + MUI + Redux Toolkit + Framer Motion + ECharts (frontend)

1. Overview

This document lists every feature currently implemented in My-Finance-Tracker, a personal finance, budgeting, and investment portfolio tracker. The project follows the phased roadmap defined in Implementation/README.md; everything below corresponds to Phase 1 (Foundation) plus a substantial set of usability and planning features (budgets, goals, notifications, recurring transactions) pulled forward from later phases.

Backend	FastAPI, SQLAlchemy, SQLite (dev), JWT auth, pytest (34 tests passing)
Frontend	React 18, TypeScript, Vite, MUI v5, Redux Toolkit, Framer Motion, ECharts
Run	Single command: ./run.sh (see RUNNING.md)
Status	Fully implemented, tested, and verified end-to-end

2. Authentication & Account

Auth

- Registration with email, password, and full name
- Password hashing with bcrypt (via passlib)
- JWT access tokens + refresh tokens, with automatic refresh-on-401 in the frontend
- Login / logout
- Default expense and income categories auto-seeded for every new user

Profile & Settings

- Update full name
- Change password (verifies current password before accepting a new one)
- Dedicated Settings page in the UI

3. Expense Management

- Full CRUD: create, list, update, delete expenses
- Categories with Need / Want classification, seeded with common defaults (Rent, Food, Groceries, Electricity, ... / Movies, Shopping, Travel, ...)
- Custom category creation
- Recurring expenses: mark an expense as monthly-recurring; missed months are automatically backfilled (idempotently) whenever the list is fetched
- Date-range and category filters (server-side query params)
- Client-side search across description and category name
- Sortable table (date, category, amount) with pagination
- Pie chart of spend by category, reflecting the current filtered view
- CSV export of the currently filtered/searched list
- Edit dialog, confirm-before-delete, and toast feedback on every action

4. Income Management

Mirrors the Expense module with an equivalent feature set:

- Full CRUD with categories (Salary, Freelancing, Dividends, Interest, Other, seeded by default; custom categories supported)
- Recurring income with automatic monthly backfill
- Date-range and category filters, search, sortable + paginated table
- Pie chart of income by source
- CSV export, edit dialog, confirm-delete, toast feedback

5. Portfolio Management

- Track holdings across stocks, ETFs, mutual funds, gold, silver, FD, PPF, NPS, crypto, and bonds
- Each holding records symbol, name, asset type, quantity, buy price/date, current price, broker, sector, exchange, target price, stop loss, and notes
- Computed per holding: invested value, current value, profit/loss, profit/loss %, and CAGR (annualized return from buy date to today)
- Full CRUD with an edit dialog and confirm-before-delete
- Search across symbol / name / sector
- Sortable table (symbol, P&L, CAGR, current value) with pagination
- Allocation pie charts by asset type and by sector
- CSV export

Note: current_price is user-editable in this phase since there is no live market data feed yet (planned for Phase 2).

6. Budget

- Set a monthly spending limit per expense category (one budget per category)
- Live tracking of amount spent this month, remaining budget, and percent used, computed from actual expense records
- Color-coded progress bar (green under 80%, amber 80-100%, red over 100%)
- Full CRUD: create, edit limit, delete

7. Goals

- Track savings goals with a name, target amount, current amount, and target date
- Computed progress percentage and monthly contribution needed to reach the goal by its target date
- Full CRUD with progress bars on each goal card

8. Notifications

- Automatic alerts when a budget crosses 80% used (warning) or 100% used (error)
- Deduplicated per budget/month so the same alert is not repeated
- Notification bell in the navbar with an unread-count badge, polling every 60s
- Mark a single notification read, or mark all as read

9. Dashboard & Analytics

- Time-of-day greeting personalized with the user's name
- KPI cards: Net Worth, Today's Profit/Loss, Monthly Income, Monthly Expenses (animated count-up on load)
- 6-month cash flow line chart (income minus expenses per month)
- 6-month expense vs income trend bar chart
- Expense allocation pie chart (current month, by category)
- Portfolio allocation pie chart (by asset type)
- Top gainers / top losers tables from current portfolio holdings
- All figures computed live from actual expense, income, and portfolio records

10. Design & User Experience

Visual design

- Custom Apple-inspired theme: Inter typeface, refined color palette, rounded cards, pill-shaped buttons
- Full light and dark mode support, including chart color theming, with a toggle in the navbar
- Frosted-glass navbar and sidebar (backdrop blur)

Animation (Framer Motion)

- Split-screen animated login/register hero panel with drifting gradient orbs
- Staggered field entrance animation on login/register forms
- Animated sliding active-page indicator in the sidebar
- Staggered card/chart entrance animations on the Dashboard
- Animated count-up numbers on KPI cards
- Cross-fade page transitions on route change

Usability infrastructure

- Toast/snackbar notifications for every create/update/delete action, success or failure
- Confirm-before-delete dialogs everywhere data can be deleted
- Loading states on all save buttons
- Skeleton loaders (replacing spinners) on Dashboard, Expenses, Income, Portfolio, Budget, and Goals
- Responsive, collapsible sidebar with a hamburger menu on mobile widths
- Global command palette (Ctrl/Cmd+K) for fast navigation
- Custom 404 page
- Per-page browser tab titles
- CSV export on Expenses, Income, and Portfolio

11. Developer & Operational Experience

- Single-command startup: ./run.sh sets up the backend venv, installs frontend deps, and launches both servers together, refusing to start if ports are already in use
- 34 backend pytest tests covering auth, expenses, income, portfolio, dashboard, budgets, goals, notifications, recurring transactions, and profile/password changes, all run against the real FastAPI app via TestClient
- Frontend type-checked end-to-end with zero TypeScript errors (tsc -b)
- RUNNING.md with full setup, troubleshooting, and Node-install instructions
- Layered backend architecture: api -> services -> repositories -> models, with Pydantic schemas at every boundary
- JWT-protected API with per-user data isolation, verified by tests

12. Not Yet Implemented (Roadmap)

The following are documented in Implementation/README.md and Implementation/Enhancements.md as future phases, and are not part of the current build:

- Live market data feeds and real-time price updates (WebSockets)
- Trading journal
- AI financial assistant, RAG document search (Ollama, ChromaDB)
- ML forecasting (expense/income/stock prediction)
- PDF/Excel report generation, dividend calendar, tax estimation
- PostgreSQL + Alembic migrations (currently SQLite with schema auto-create, dev-only)
- CI/CD pipeline, structured logging, error tracking, metrics/monitoring
- Two-factor authentication, refresh-token revocation, rate limiting
- Progressive Web App (offline mode, push notifications)